Classification of Birds by Sound

By Jennifer Poling

Introduction

This project uses deep learning to classify Seattle birds by their sounds. The question being investigated is, “Can a convolutional neural network model predict the bird species from spectrograms of the bird’s sounds?” This model can be used in ecology and environmental science.

A convolutional neural network (CNN) is built to predict which bird species made different sounds. The data used in this project is made up of two second long spectrograms of bird sounds labeled with the species of bird. The data comes from the birdcall competition data on Kaggle [1]. The data that I used was preprocessed and can be accessed from GitHub [2]. The test data can also be accessed from GitHub [3].

Theoretical Background

The model used in this project is a convolutional neural network (CNN). A CNN is a deep learning model. Neural networks have an input layer. The number of features in the input layer is determined by the number of predictors in the data. There are weights that connect the input layer to the hidden layer. The connections from the hidden layer to the output layer are called biases. The parameters of the weights and biases can be tuned as part of training the model.

There can be many parameters in a neural network; there are often more parameters than there is data. Therefore, neural networks are prone to overfitting. One way to combat over fitting is to use regularization. Dropout regularization is often used when building neural networks. Dropout regularization tells the model to ignore a certain percentage of nodes in a layer.

Rectified linear unit, or ReLU, activation is a popular choice for the activation function in neural networks and was used in this project. Sigmoid activation was used for the binary model in this project. It is often used for binary projects because the results will be between zero and one, as is necessary for a binary model that is predicting either zero or one. Softmax activation was used for the multi-class model in this project because it causes all probabilities to add to one.

Convolutional neural networks are used for image classification. A CNN was used in this project because the audio data was turned into a spectrogram, which is a visual representation of the audio data. Thus, a CNN could be used to predict the spectrogram image.

Convolutional neural networks use a convolutional filter in the convolution layer. Then each convolution layer is followed by a pooling layer. A convolutional filter basically slides a filter across an image, matching similar features in the image to features in the filter, and computes a smaller set of features using dot product multiplication. Hyperparameters that can be tuned in the convolution layer include the size of the filter and the number of filters applied. The size of the filter should be determined based on the size of the image so that the filter is much smaller than the image. The convolution layer produces a lot of data, and the pooling layer shrinks the data back to a smaller size.

The models in this project were evaluated using accuracy. This is because the models were classification models. The purpose of the models was to accurately identify the bird species from the spectrogram of the bird sounds.

Methodology

The data was normalized and split into train and test sets. Because the minimum value was -80, the data was normalized by dividing each item by 80. This meant that the normalized values were between negative one and approximately zero.

In order to create a binary model, the data for the Bewick's Wren and Spotted Towhee were separated from the rest of the data. These species were chosen because there was a similar amount of data for each of them. They were chosen to avoid a problem with class imbalance. After separating out the data for the Bewick's Wren and Spotted Towhee, the data set had 281 items.

Because the data set was small a small CNN model was specified using three convolution layers with filters of size 16, 32, and 64 and a dropout rate of 0.5. The model was compiled using binary crossentropy for loss, since it's a binary model, optimizer type adam, and the metric accuracy. Accuracy was chosen as the metric because the classification model is predicting one species of bird or another, and the goal is accurate predictions. The model was trained with 15 epochs. Next the same model was used with more epochs and early stopping. Next the same model was used with higher patience in the early stopping settings. Next a larger CNN model was built. This model still had three convolution layers but with filters of size 32, 64, and 128. Next a smaller dropout rate was used on the smaller binary model.

The next portion of the project was to make a multi-class CNN. Labels were one-hot encoded. A CNN model was specified using three convolution layers with filters of size 32, 64, and 128, a dropout rate of 0.5, and 15 epochs. Next a smaller model was specified using three convolution layers with filters of size 32, 64, and 64, a dropout rate of 0.6, and 10 epochs. Next the smaller model was tried again but with a dropout rate of 0.5. Then class weights were computed and

used in training. Next a different formula was used for class weights rather than raw class weights.

Finally, the top performing model used to predict the bird species in the external test data. The external test data was split into two second long chunks of sound. These two second chunks were turned into spectrograms, which were normalized the same way the original data was normalized. Then the model predicted which species was represented by each spectrogram.

Results

The first binary model, using three convolution layers with filters of size 16, 32, and 64 and a dropout rate of 0.5, had a test accuracy of 73.68%. A confusion matrix of this model is included below.

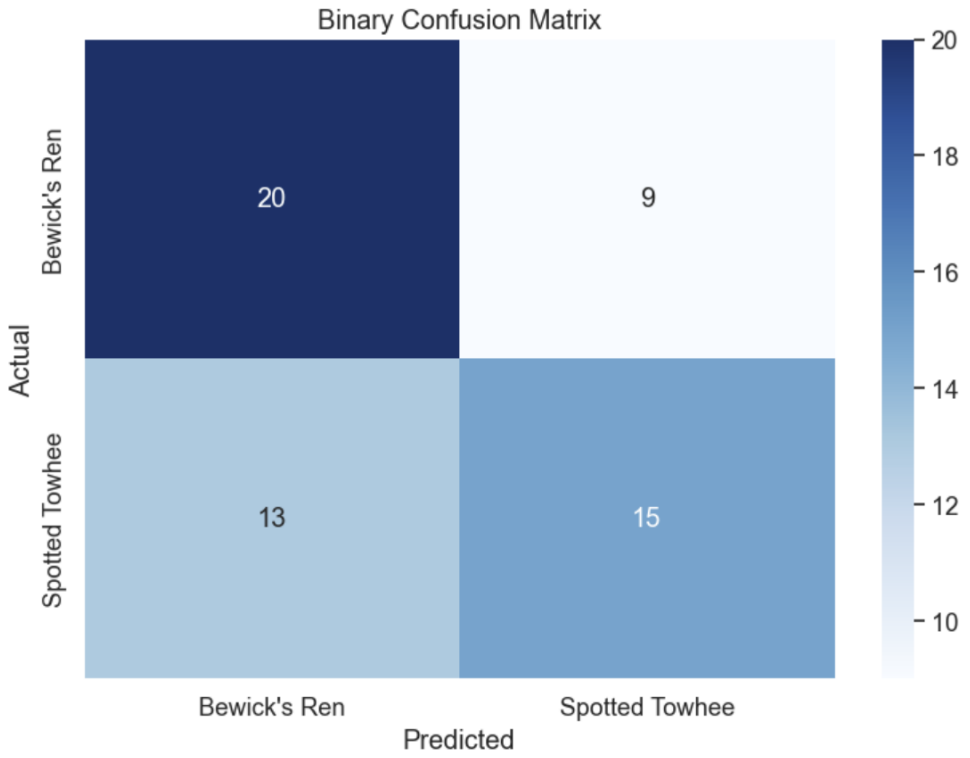

Figure 1 Binary Confusion Matrix

The second binary model, which was the same model but with more epochs and early stopping, had a test accuracy of 49.12%.

The third binary model, which was the same as the second model but with higher patience, had a test accuracy of 61.40%.

The larger binary model had a test accuracy of 52.63%.

The small binary model with a smaller dropout rate had a test accuracy of 54.39%.

All of the model results are included in Figure 2 below.

Binary Model Results	
Model	Test Accuracy
small model	73.68%
early stopping	49.12%
higher patience	61.40%
smaller dropout	54.39%
large model	52.63%

Figure 2 Binary Model Results

The first multi-class model had a test accuracy of 42.82%. The next model had a test accuracy of 43.58%. The next model had a test accuracy of 45.59%. The model using raw class weights had a test accuracy of 7.30%. The model using adjusted class weights had a test accuracy of 44.08%. All of the multi-class model results are included in Figure 3 below.

Multi-Class Model Results	
Model	Test Accuracy
base model	42.82%
smaller model	43.58%
smaller dropout	45.59%
raw class weights	7.30%
adjusted class weights	44.08%

Figure 3 Multi-Class Model Results

The confusion matric for the top performing multi-class model is included in Figure 4 below.

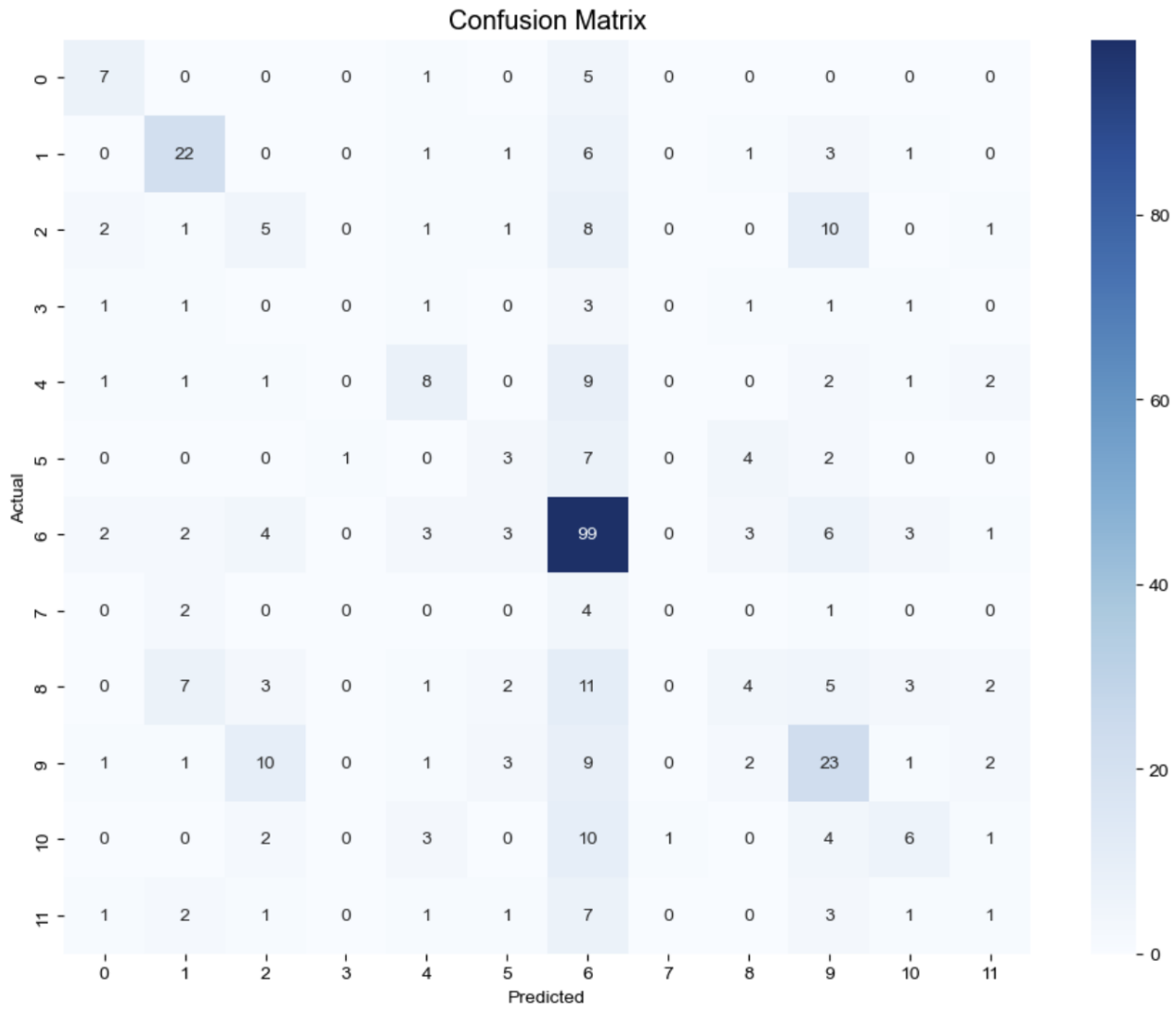

Figure 4 Multi-Class Confusion Matrix

Confusion Matrix Legend	
Bird Species	Number
American Crow	0
American Robin	1
Bewick's Ren	2
Black-capped Chickadee	3
Dark-eyed Junko	4
House Finch	5
House Sparrow	6
Northern Flicker	7
Red-winged Blackbird	8
Song Sparrow	9
Spotted Towhee	10
White-crowned Sparrow	11

Figure 5 Multi-Class Model Confusion Matrix Legend

The top performing multi-class model was used to predict the bird species represented in the external test data. The model predicted that the first external test data included sounds of the House Sparrow, Red-Winged Blackbird, and American Robin. The model predicted that the second external test data included sounds of only the Red-Winged Blackbird. The model predicted that the third external test data included sounds of the House Sparrow and the Red-Winged Blackbird.

Discussion

The small binary model has a strong test accuracy of 73.68%. It looked like the training accuracy might be able to improve with more epochs. At the same time, the loss was decreasing. Therefore, the same model was tried again with more epochs and early stopping. That model finished after four epochs, possibly because the data set was small and the validation metrics were noisy. This was why a higher patience setting was used in the early stopping metrics. The test accuracy improved, but it was not as high as the test accuracy for the original model.

The larger binary model overfit the small data set, which resulted in a lower test accuracy. The smaller binary model was determined to be the stronger model. Next, a smaller dropout rate was used with the smaller model. The smaller dropout rate did not improve the model, likely due to the small data set. The original small model was determined to be the strongest model.

The first multi-class model appeared to be overfitting. This could be because multi-class classification is harder for the model to learn. The second model with a higher dropout and fewer epochs appeared to be underfitting, so the next model was kept small but again used the original dropout rate. The next model had class weights computed and used in training to try to account for the class imbalance. Raw class weights caused what seems to be random guessing by the model, so next changing the formula for the class weights was tried. The top performing model was the smaller model with the 0.5 dropout rate.

Regarding the limitations I ran into in this homework: I found it hard to get started. I had never worked with audio files before. I hadn't used HDF5 format before. It took me a while to figure out how to access the data. It didn't take that long to train the models. After reading the assignment, I was expecting it to take a lot longer. The longest training time I experienced was just over three minutes. I might have thought this was a long time if I hadn't been expecting it to take longer and if I hadn't been taking the big data systems course at the same time. This quarter I have often had to wait for work to finish.

Another limitation I ran into was the model results changing. I included both code1 and code2 in my GitHub. After I finished most of my report, I went back to my first binary model to make a confusion matrix. Because I used the same name for all of the models, I had to specify the

model again and refit it. I got a much lower test accuracy when I did this. I set a seed and ran it again. I got a test accuracy closer to my original, but still not as good. The test accuracies in my report are from code1, and the binary model confusion matrix is from code 2. For future models, setting an overall seed at the beginning might help prevent this. However, part of the issue is due to the nature of convolutional neural networks and the fact that I was using a small data set. The network may learn slightly different things when it runs than it had the previous time, especially when the data set is small.

I didn't notice one species of bird getting confused for another one so much as one getting predicted the most. The House Sparrow was predicted the most frequently. It was correctly predicted the most frequently, but it was also predicted as every other species. I think this happened because it had the most examples in the data set. I wasn't able to build a model that was exceptionally good at predicting the bird species, and I therefore think that the model predicted the House Sparrow the most often because that increased the odds that it would be right.

Other models that could have been used to perform this task include random forest and support vector machines, but they would have required more feature engineering. A neural network makes sense for this task because this is a task that convolutional neural networks are very good at.

Better results could have been obtained using a pre-trained model. Companies who have made pre-trained models available put a lot of time, money, and expertise into creating these models. Students building from scratch are unlikely to match the results they could obtain building on a pre-trained model.

Conclusions

The purpose of this project was to determine whether a convolutional neural network could predict bird species using spectrograms of the birds' sounds. The models built during this project struggled to predict the bird species correctly. One way to build on this project would be to use a pre-trained model as the basis for a model to predict bird species. This approach is likely to produce a model that is more successful at predicting bird species. Another way to build on this project would be to collect more data. More data, especially more balanced data, could produce a more robust model.

There are some broader impacts and real-world implications of this work. A strong model that is good at predicting bird species from their sounds could be used in environmental work. It could be used to track the types of birds in an area. Birds can be hard to spot in trees and bushes. It can be easier to make recordings of the birds in an area to determine which birds are there. A

strong model could also be used to measure the health of a habitat by determining the number of bird species present.

References

[1] Vopani, “Xeno-Canto Bird Recordings Extended (A-M),” Kaggle, n.d. [Online]. Available: https://www.kaggle.com/datasets/rohanrao/xeno-canto-bird-recordings-extended-a-m (accessed May 19, 2026).

[2] A. Mendible, “bird_spectrograms.hdf5,” GitHub, n.d. [Online]. Available: https://github.com/mendible/5322/blob/main/Homework%203/bird_spectrograms.hdf5 (accessed May 19, 2026).

[3] A. Mendible, “test_birds,” GitHub, n.d. [Online]. Available: https://github.com/mendible/5322/tree/51bd4705bf06d4f46e1848f9261da9b2db3333c0/Homework%203/test_birds (accessed May 19, 2026).

[4] Python Software Foundation, “Python,” v. 3.14, 2025. [Computer software]. Available: https://www.python.org/ (accessed May 19, 2026).

[5] Project Jupyter, “Jupyter Notebook,” v. 7.0, 2023. [Computer software]. Available: https://jupyter.org/ (accessed May 19, 2026).

[6] P. Nandi, “CNNs for Audio Classification,” Towards Data Science, March 2021. [Online]. Available: https://towardsdatascience.com/cnns-for-audio-classification-6244954665ab/ (accessed May 19, 2026).

[7] “Image Resizing using OpenCV | Python,” Geeks for Geeks, October 2025. [Online]. Available: https://www.geeksforgeeks.org/python/image-resizing-using-opencv-python/ (accessed May 19, 2026).